

PARALLEL COMPUTING LABORATORY

THE AMDAHL REALITY GAP

Empirical Speedup Fitting, Micro-Architectural Bottlenecks,
and Loop Scheduling in OpenMP

Student Full Name	Mardan Ibragimov
Student ID	230103054
Date	24 September 2026
Operating System	Windows
CPU	12th Gen Intel(R) Core(TM) i7-12700H
Physical / Logical Threads	14 physical cores / 20 logical threads
Workload N	13,054,000

All timing values in this report are transcribed from the student's local Windows/OpenMP execution shown in the appendix screenshots.

1. Machine, workload and measurement setup

The workload is tied to Student ID 230103054. The last four digits are 3054, therefore $N = 10,000,000 + (3054 \times 1,000) = 13,054,000$.

The program was compiled with GCC using -O2 and -fopenmp. For each benchmark, the first run was treated as a cold/warm-up run, while Run 2 and Run 3 were averaged for the reported time. Wall-clock timing was performed with OpenMP timing as required by the worksheet.

Item	Observed value
CPU	12th Gen Intel(R) Core(TM) i7-12700H
Physical cores	14
Logical/OpenMP max threads	20
Cache-line size used in analysis	64 bytes (worksheet/x86 assumption)
Target workload N	13,054,000
Sequential T_seq	4.218500 s
Verification checksum	62,455,498
Maximum stopping time	688
Numbers with >100 steps	10,516,340

Table 1. Machine and workload summary.

2. Sequential baseline and OpenMP scaling

The sequential baseline averaged 4.218500 s from Run 2 and Run 3. The measured checksum was 62,455,498, the maximum Collatz stopping time was 688, and 10,516,340 inputs required more than 100 steps.

k	Cold	Run 2	Run 3	Avg T_k	S_emp	S_theo	Delta
1	2.531	7.627	7.918	7.7725	0.5427	1.0000	+0.4573
2	3.884	4.008	4.080	4.0440	1.0432	1.0432	-0.0000
4	1.896	1.889	1.909	1.8990	2.2214	1.0662	-1.1552
8	1.002	0.958	0.939	0.9485	4.4475	1.0780	-3.3695
14	0.927	0.942	0.937	0.9395	4.4902	1.0832	-3.4070
16	0.925	0.943	0.932	0.9375	4.4997	1.0841	-3.4156
20	0.934	0.939	0.993	0.9660	4.3670	1.0853	-3.2817

Table 2. Raw scaling measurements and one-point Amdahl fit. $\Delta = S_{theo} - S_{emp}$.

From $k=2$, $S_{emp}(2) = 1.0432$ and $p = 2[1 - 1/S_{emp}(2)] = 0.082731$. This is a very low fitted parallel fraction because the $k=2$ measurement is close to the sequential baseline. Higher thread counts, however, produced much larger empirical speedups. Therefore the one-point Amdahl fit under-predicts the $k=4$ to $k=20$ measurements and gives negative Delta values there. This should be read as a limitation of fitting p from a single, noisy/overhead-sensitive $k=2$ point rather than as a claim that the physical machine violates Amdahl's Law.

3. Empirical vs. theoretical speedup

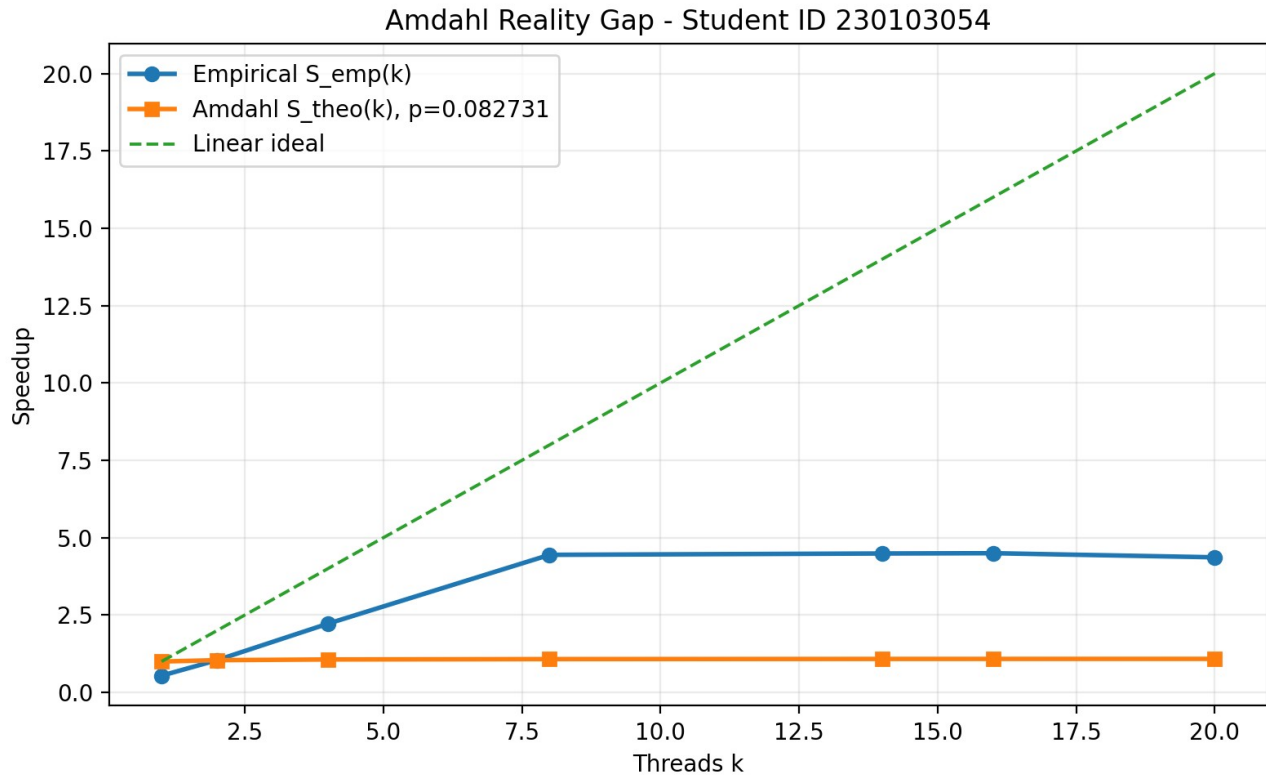


Figure 1. Empirical speedup, Amdahl estimate derived from $k=2$, and linear ideal.

The empirical curve rises sharply between 2 and 8 threads and then plateaus around 4.4-4.5x. At 20 logical threads, speedup falls slightly to 4.367x. This is consistent with a physical-core ceiling: after the physical cores are busy, SMT threads compete for shared execution resources instead of providing another full core of throughput.

4. False sharing experiment

Variant	Threads	Avg time (s)	Throughput (iter/s)	Penalty ratio
Naive hits[tid]++	14	0.999500	13,060,530	1.0566x
OpenMP reduction	14	0.946000	13,799,154	1.0000x

Table 3. False-sharing penalty at 14 threads.

The naive per-thread counter array took 0.999500 s, compared with 0.946000 s for the reduction version. The measured penalty ratio was 1.0566x, so the naive false-sharing variant was about 5.7% slower in this run.

5. Loop scheduling experiment

OpenMP scheduling clause	Chunk	Avg time (s)
schedule(static)	default	0.838000
schedule(static, 1000)	1000	0.438500
schedule(dynamic, 100)	100	0.999000
schedule(dynamic, 10000)	10000	0.943000
schedule(guided)	adaptive	0.995500

Table 4. Scheduling results at 14 threads.

The fastest measured policy was schedule(static, 1000) at 0.438500 s. Fine-grained dynamic(100) was the slowest at 0.999000 s, which indicates that queue/lock scheduling overhead outweighed the benefit of finer load balancing for this workload on this machine. Larger dynamic chunks reduced queue activity but still remained slower than static(1000).

6. Technical analysis / defense answers

Q1. Micro-architectural root cause of false sharing

Variant 1 stores the per-thread counters next to each other in a small array. With a 64-byte cache line, counters belonging to several threads can occupy the same line. When different cores repeatedly update their own counters, the MESI/MOESI coherence mechanism must invalidate copies of that line and transfer write ownership between cores. The variables are logically independent, but the hardware sees writes to the same cache line, so extra cache-coherence and interconnect traffic is generated. In this run the naive variant took 0.999500 s while the reduction version took 0.946000 s, a 1.0566x penalty (about 5.7% slower).

Q2. Hyperthreading (SMT) saturation and physical-core ceiling

Scaling did not continue linearly after the physical-core count. SMT threads on one core share the front end/decode path, execution ports, caches, load/store resources and memory bandwidth. The measured speedup was 4.4902x at 14 physical cores and 4.3670x at 20 logical threads, which is about a 2.7% decrease. On this workload, enabling all logical threads therefore added contention rather than another full set of execution resources.

Q3. Empirical vs. theoretical Amdahl discrepancy

The required $k=2$ fit gives $p = 0.082731$. Because the measured $k=2$ speedup was only 1.0432x, this one-point estimate predicts a theoretical speedup of only about 1.08x at high k , while the later measured values are around 4.4-4.5x. The main reason the curves separate so strongly is that the p estimate is very sensitive to the $k=2$ timing and its OpenMP/runtime overhead. In addition, the basic Amdahl equation does not model real hardware effects. Two concrete omitted factors observed in this lab are cache-coherence/false-sharing traffic and non-uniform Collatz work combined with scheduling/synchronization overhead. Memory-bandwidth pressure and SMT resource contention also matter as k increases.

Q4. Scheduling trade-off dilemma

The measured averages were dynamic(100) = 0.999000 s, dynamic(10000) = 0.943000 s, static = 0.838000 s, static(1000) = 0.438500 s and guided = 0.995500 s. The 100-iteration dynamic chunks caused the most queue activity and were slower than the 10,000-iteration dynamic chunks, so on this machine the overhead of fine-grained dynamic scheduling outweighed its load-balancing benefit. Static scheduling has low runtime scheduling overhead but may suffer if work is distributed unevenly. In this run, static(1000) gave the best balance between low overhead and distribution of the irregular Collatz iterations.

7. Submission checklist

- hw_info.txt - raw Windows CPU specification output
- collatz.c - OpenMP source code
- results.csv - raw benchmark data
- speedup_plot.png - speedup graph
- analysis.pdf - this written defense/report
- meta.txt and tables.txt - supporting benchmark summaries

Recommended Git folder name: lab1_230103054. Keep the local results.csv and hw_info.txt generated on this laptop; they are the machine-specific evidence required by the worksheet.

8. Git repository packaging

Before pushing to Git, place this PDF in the same lab1_230103054 folder that already contains the files produced by the local run. The final folder should keep the machine-generated results intact.

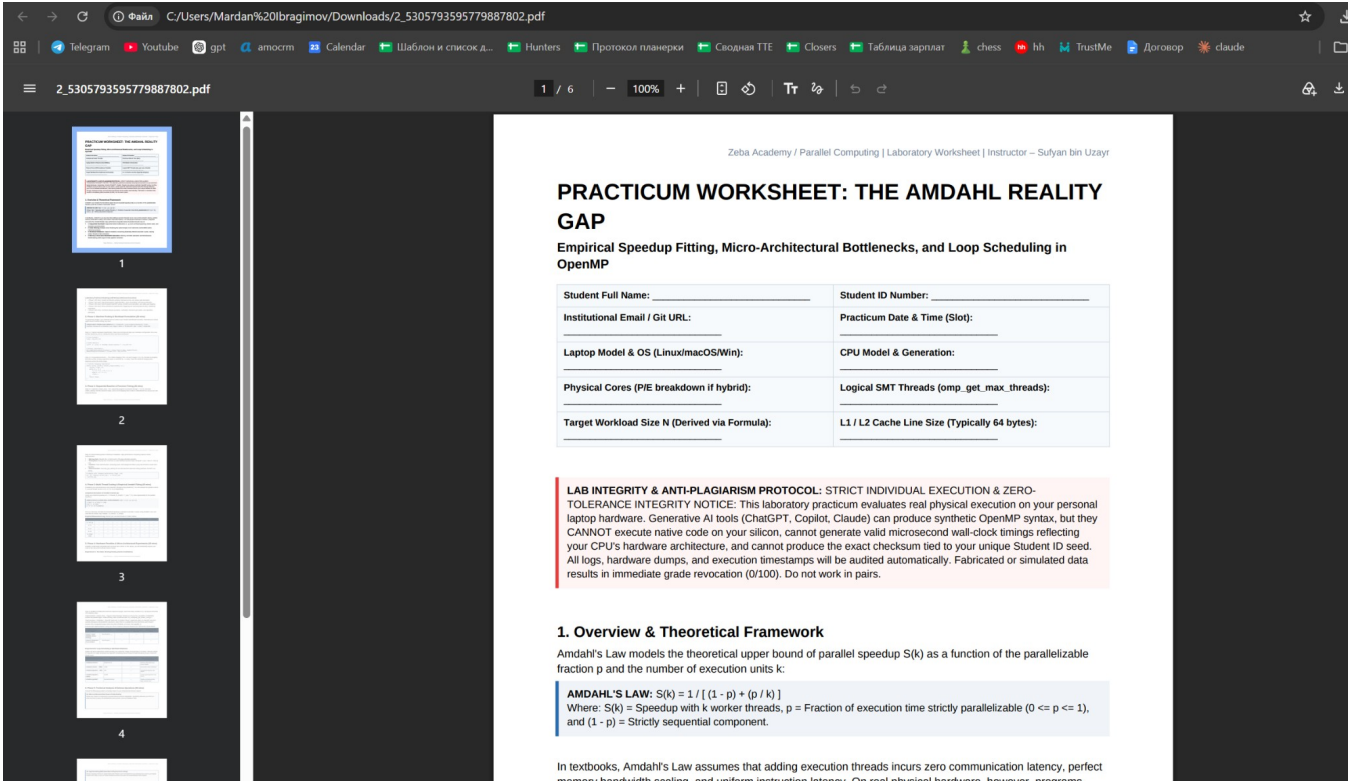
```
lab1_230103054/  
  collatz.c  
  hw_info.txt  
  results.csv  
  speedup_plot.png  
  analysis.pdf  
  meta.txt  
  tables.txt  
  analysis.txt    (optional supporting text)
```

If the repository is new, open PowerShell inside the folder and run the following commands after creating the repository on GitHub/GitLab:

```
git init  
git add .  
git commit -m "Parallel Computing Lab 1 - 230103054"  
git branch -M main  
git remote add origin <YOUR_REPOSITORY_URL>  
git push -u origin main
```

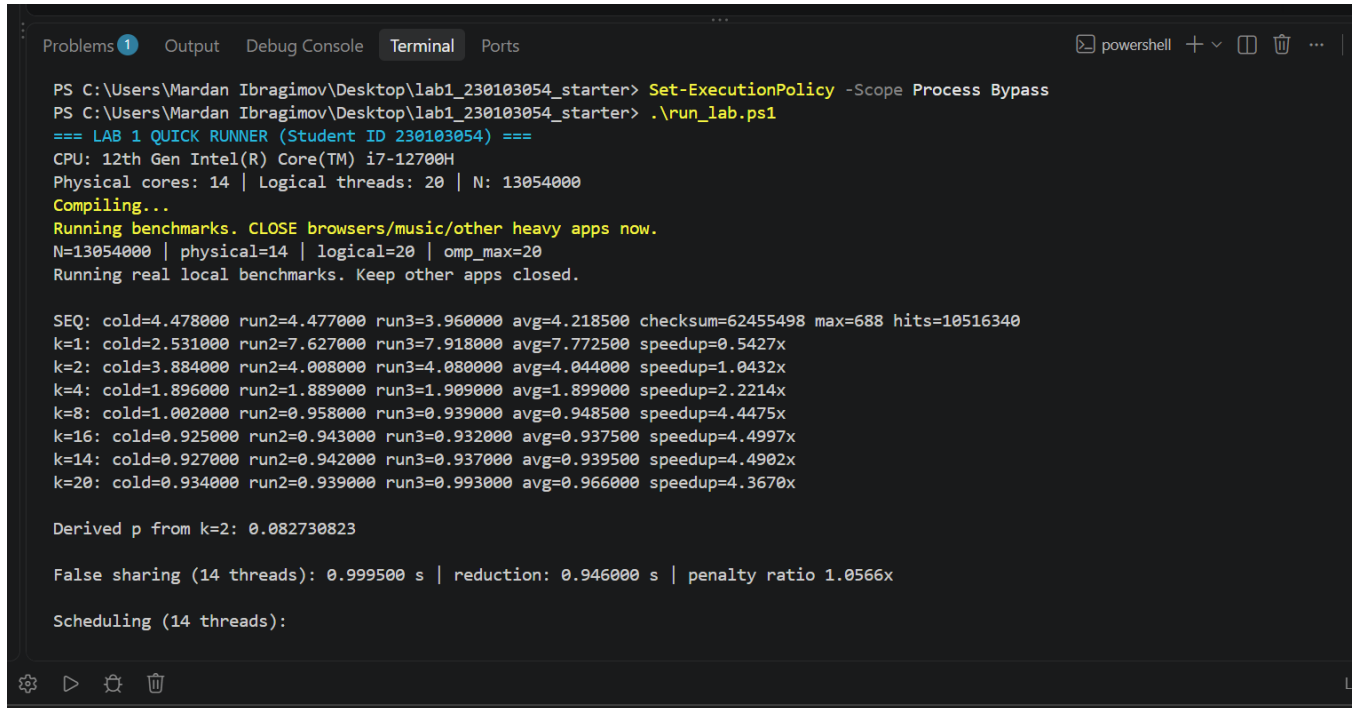
After the push, copy the repository URL into the Institutional Email / Git URL field if your instructor expects it on the worksheet or submission form.

Appendix A1. Execution evidence



Reference: original laboratory worksheet opened locally.

Appendix A2. Execution evidence



```
PS C:\Users\Mardan Ibragimov\Desktop\lab1_230103054_starter> Set-ExecutionPolicy -Scope Process Bypass
PS C:\Users\Mardan Ibragimov\Desktop\lab1_230103054_starter> .\run_lab.ps1
=== LAB 1 QUICK RUNNER (Student ID 230103054) ===
CPU: 12th Gen Intel(R) Core(TM) i7-12700H
Physical cores: 14 | Logical threads: 20 | N: 13054000
Compiling...
Running benchmarks. CLOSE browsers/music/other heavy apps now.
N=13054000 | physical=14 | logical=20 | omp_max=20
Running real local benchmarks. Keep other apps closed.

SEQ: cold=4.478000 run2=4.477000 run3=3.960000 avg=4.218500 checksum=62455498 max=688 hits=10516340
k=1: cold=2.531000 run2=7.627000 run3=7.918000 avg=7.772500 speedup=0.5427x
k=2: cold=3.884000 run2=4.008000 run3=4.080000 avg=4.044000 speedup=1.0432x
k=4: cold=1.896000 run2=1.889000 run3=1.909000 avg=1.899000 speedup=2.2214x
k=8: cold=1.002000 run2=0.958000 run3=0.939000 avg=0.948500 speedup=4.4475x
k=16: cold=0.925000 run2=0.943000 run3=0.932000 avg=0.937500 speedup=4.4997x
k=14: cold=0.927000 run2=0.942000 run3=0.937000 avg=0.939500 speedup=4.4902x
k=20: cold=0.934000 run2=0.939000 run3=0.993000 avg=0.966000 speedup=4.3670x

Derived p from k=2: 0.082730823

False sharing (14 threads): 0.999500 s | reduction: 0.946000 s | penalty ratio 1.0566x

Scheduling (14 threads):
```

Local execution: CPU detection, sequential baseline, thread scaling, derived p and false-sharing result.

This screen captures the key benchmark values used in Tables 1-3, including CPU/core detection, sequential baseline, all thread-count timings, the fitted p value and the false-sharing comparison.

Appendix A3. Execution evidence

```

k=8: cold=1.002000 run2=0.958000 run3=0.939000 avg=0.948500 speedup=4.4475x
k=16: cold=0.925000 run2=0.943000 run3=0.932000 avg=0.937500 speedup=4.4997x
k=14: cold=0.927000 run2=0.942000 run3=0.937000 avg=0.939500 speedup=4.4902x
k=20: cold=0.934000 run2=0.939000 run3=0.993000 avg=0.966000 speedup=4.3670x

Derived p from k=2: 0.082730823

False sharing (14 threads): 0.999500 s | reduction: 0.946000 s | penalty ratio 1.0566x

Scheduling (14 threads):
  static      avg=0.838000 s
  static_1000 avg=0.438500 s
  dynamic_100 avg=0.999000 s
  dynamic_1000 avg=0.943000 s
  guided      avg=0.995500 s

DONE: results.csv and meta.txt created.
Downloading: ..... ✓
Extracting: ..... ✓
py : Python install manager was successfully updated to 26.3.
C:\Users\Mardan Ibragimov\Desktop\lab1_230103054_starter\run_lab.ps1:28 знак:5
+   py -c "import matplotlib" 2>$null
+   ~~~~~
+ CategoryInfo          : NotSpecified: (Python install ...pdated to 26.3.:String) [], RemoteException

```

Local execution: scheduling results and completion of results.csv/meta.txt generation.

This screen captures the scheduling averages and confirms that results.csv and meta.txt were produced by the local run.

Appendix A4. Execution evidence

```

Traceback (most recent call last):
  File "C:\Users\Mardan Ibragimov\Desktop\lab1_230103054_starter\make_outputs.py", line 84, in <module>
    hw_text=HW.read_text(encoding='utf-16', errors='ignore') if HW.exists() else ''
    ~~~~~^~~~~~
  File "C:\Users\Mardan Ibragimov\AppData\Local\Python\pythoncore-3.14-64\Lib\pathlib\__init__.py", line 788, in read_text
    return f.read()
    ~~~~~^~
  File "<frozen codecs>", line 325, in decode
  File "C:\Users\Mardan Ibragimov\AppData\Local\Python\pythoncore-3.14-64\Lib\encodings\utf_16.py", line 67, in _buffer_decode
    raise UnicodeDecodeError("utf-16", input, 0, 2, "Stream does not start with BOM")
UnicodeDecodeError: 'utf-16' codec can't decode bytes in position 0-1: Stream does not start with BOM
PS C:\Users\Mardan Ibragimov\Desktop\lab1_230103054_starter> (Get-Content .\make_outputs.py -Raw).Replace("hw_text=HW.read_text(encoding='utf-16', errors='ignore') if HW.exists() else '', "hw_text=HW.read_text(encoding='utf-8', errors='ignore') if HW.exists() else ''") | Set-Content .\make_outputs.py -Encoding utf8
PS C:\Users\Mardan Ibragimov\Desktop\lab1_230103054_starter> py .\make_outputs.py
Created: speedup_plot.png, analysis.txt, tables.txt
PS C:\Users\Mardan Ibragimov\Desktop\lab1_230103054_starter> mkdir lab1_230103054
>> Copy-Item hw_info.txt, collatz.c, results.csv, speedup_plot.png, analysis.txt, tables.txt, meta.txt -Destination .\lab1_230103054\
>> tar -czvf lab1_230103054.tar.gz lab1_230103054

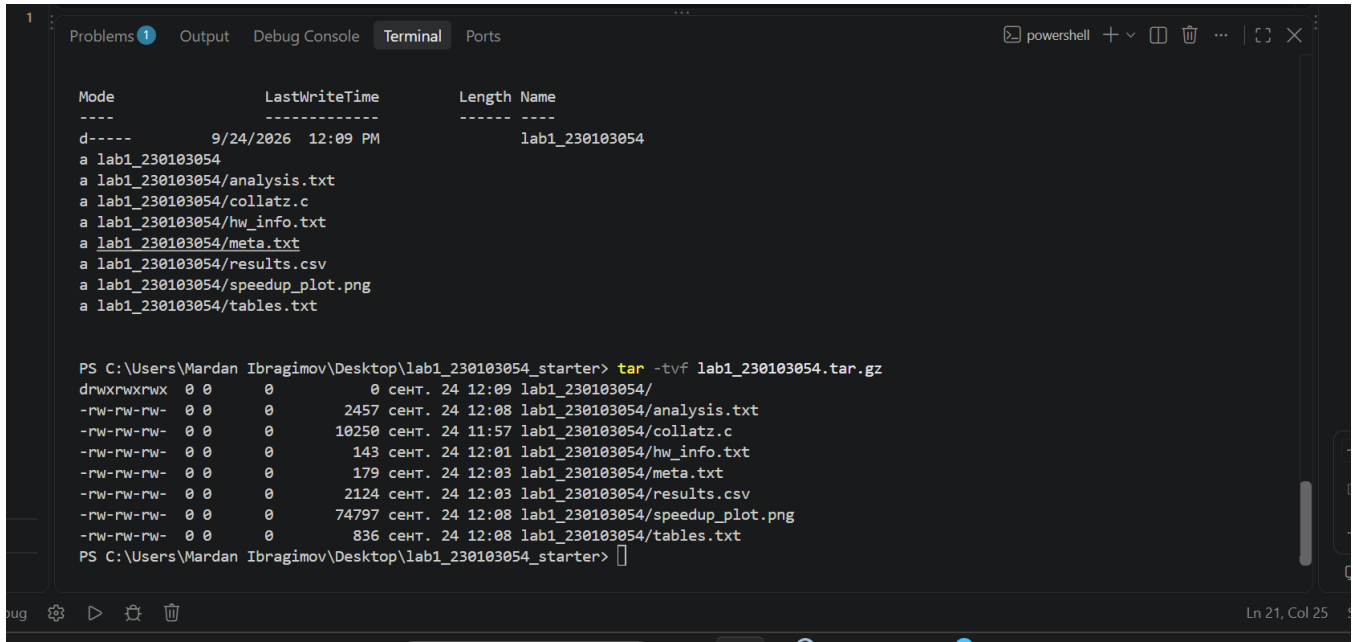
Катанор: C:\Users\Mardan Ibragimov\Desktop\lab1_230103054_starter

```

Local output generation: encoding issue was corrected and report artifacts were created successfully.

The initial Python post-processing failed only when reading hw_info.txt with the wrong text encoding. After changing the reader to UTF-8, speedup_plot.png, analysis.txt and tables.txt were created successfully; the benchmark data itself had already completed and was not re-generated.

Appendix A5. Execution evidence



The screenshot shows a PowerShell terminal window with the following content:

```
Mode                LastWriteTime         Length Name
----                -
d-----          9/24/2026 12:09 PM                lab1_230103054
a lab1_230103054
a lab1_230103054/analysis.txt
a lab1_230103054/collatz.c
a lab1_230103054/hw_info.txt
a lab1_230103054/meta.txt
a lab1_230103054/results.csv
a lab1_230103054/speedup_plot.png
a lab1_230103054/tables.txt

PS C:\Users\Mardan Ibragimov\Desktop\lab1_230103054_starter> tar -tvf lab1_230103054.tar.gz
drwxrwxrwx  0 0      0      0 сент. 24 12:09 lab1_230103054/
-rw-rw-rw-  0 0      0      2457 сент. 24 12:08 lab1_230103054/analysis.txt
-rw-rw-rw-  0 0      0     10250 сент. 24 11:57 lab1_230103054/collatz.c
-rw-rw-rw-  0 0      0      143 сент. 24 12:01 lab1_230103054/hw_info.txt
-rw-rw-rw-  0 0      0      179 сент. 24 12:03 lab1_230103054/meta.txt
-rw-rw-rw-  0 0      0      2124 сент. 24 12:03 lab1_230103054/results.csv
-rw-rw-rw-  0 0      0     74797 сент. 24 12:08 lab1_230103054/speedup_plot.png
-rw-rw-rw-  0 0      0      836 сент. 24 12:08 lab1_230103054/tables.txt
PS C:\Users\Mardan Ibragimov\Desktop\lab1_230103054_starter>
```

Archive verification: final lab folder contents listed by tar -tvf.

The tar listing confirms that the submission archive contained the source, hardware information, results, graph and analysis/supporting text files.